

Embedded Systems Architectures - Laboratory Guide 10

TFT Graphical Interfaces

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Familiarization with the SS7735S driver for the Adafruit 0.96" TFT display
- Familiarization with a basic graphics library for drawing geometric shapes, lines, and text
- Building multi-screen graphical interfaces for IoT devices

2. Introduction

In the context of the Internet of Things, it is increasingly important that devices allow for clear and efficient interaction with users. Human-Machine Interfaces (HMIs) are fundamental to making the information generated by intelligent systems more accessible and understandable.

This guide aims to demonstrate this interaction using the ESP32-C6 with an Adafruit 0.96-inch TFT screen. Through this combination, the intention is to create a simple, functional visual interface that presents real-time information, improves interaction between the user and the device, and enhances access to the device's context.

3. Practical Work

The practical work in this lesson is based on the driver and graphics library available in the ST7735-ESP32-C6 repository. Additionally, the technical data sheet for the Adafruit 160x80 0.96" display, available on the course e-learning page, should be consulted.

The ST7735-ESP32-C6 repository provides a modular, efficient foundation for communicating with the TFT screen via the SPI interface, including functions for initialization, color control, drawing basic shapes, text display, and updating screen content. The code structure also allows for easy extension, facilitating the creation of interactive menus, real-time data graphs, or sensor status indicators, all of which are typical elements of a functional HMI.

3.1 Basic Operation

The first step in this guide is to mount the TFT display on the whiteboard and connect it to the ESP32-C6. For this purpose, use the circuit in Figure 1. Note that the brightness pin (LIT) is connected to a generic GPIO of the ESP32. The chip select for the SD card reader is connected to 3.3V to ensure that SPI communication occurs exclusively with the TFT.

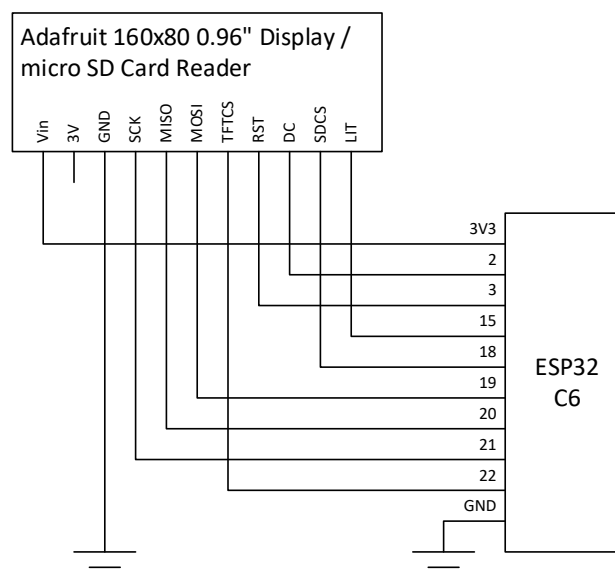


Figure 1 - Wiring diagram of the Adafruit 0.96" TFT display.

After making the indicated connections, download the example application used in this guide. To do this, run the following command in your terminal. Note that you must have previously installed the git application on your system.

```
~$ git clone https://github.com/T4V4RES/Driver-ST7735-ESP32-C6.git
```

Open the project in Visual Studio Code and change the Device Target from ESP32 to ESP32-C6.

Question 1: Analyze the repository, studying the README file, the driver, and the graphics library. Familiarize yourself with the example as well. Compile the program and download it to the board. You should see the following event log in the terminal. Is the sequence of operations visible on the TFT display as expected?

```
...
I (256) MAIN_APP: =====
I (256) MAIN_APP: Iniciando Display ST7735S Demo
I (256) MAIN_APP: =====
I (266) ST7735: ST7735 Driver - Adafruit Mini TFT 0.96
I (266) ST7735: Pinos: MOSI=19 CLK=21 CS=22 DC=2 RST=3 BL=15
I (276) ST7735: Backlight ON
I (276) ST7735: SPI @ 8 MHz
I (1386) ST7735: Display OK: 160x80 pixels
I (1386) MAIN_APP: Display inicializado com sucesso!
I (1386) MAIN_APP: A testar cores...
I (6466) MAIN_APP: A desenhar linhas...
I (7486) MAIN_APP: A escrever texto...
I (8616) MAIN_APP: A desenhar linhas...
I (9636) MAIN_APP: A escrever texto...
```

Question 2: Analyze the README file and test examples 1, 2, and 3 indicated therein. Perform the tests with different colors, font sizes, rectangle sizes and ratios, and line thickness parameters. In this question, you should familiarize yourself with the implementation of the different functions of the graphics library.

Question 3: Assuming that you want to emulate the operation of an RGB LED on the TFT display to indicate the status of an operation according to the following

definition, implement a program that displays a fixed radius circle centered on the screen and that alternates between the 3 possible colors once per second.

```
{  
  {"ok", green},  
  {"error", red},  
  {"fault", yellow}  
}
```

Question 4: Develop a graphical interface to display ID information, temperature, humidity, and operating status of an IoT device. Consider that the device's displayed status follows the thresholds: {cold: < 22 degrees}, {alert: > 30 degrees}, and {normal: all other cases}. The graphical interface to be built should use random temperature and humidity data and update the displayed information every second. This graphical interface should follow the layout shown in Figures 2, 3, and 4.

Question 5: Integrate the developed interface with the DHT20 temperature and humidity sensor used in previous classes. Test it by heating the sensor and verifying that the device's state changes accordingly.

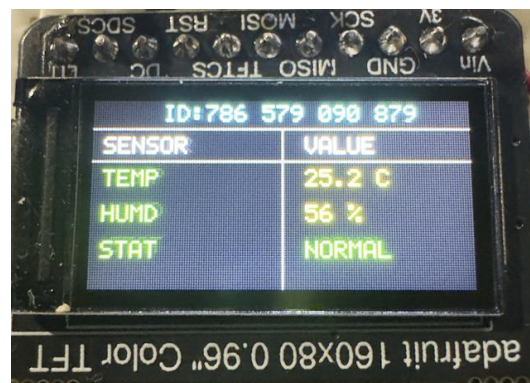


Figure 2 - HMI layout for the NORMAL state.

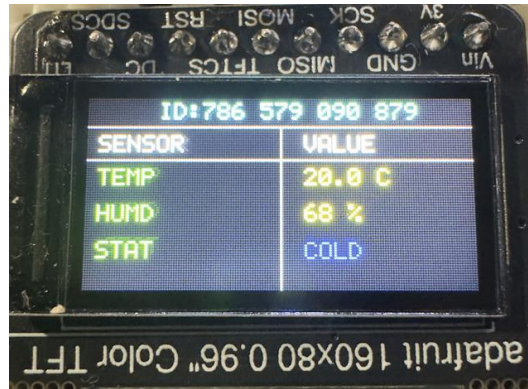


Figure 3 - HMI layout for the COLD state.



Figure 4 - HMI layout for the ALERT state.

Question 6: Using a push button, create a graphical interface that toggles between two screens. The default screen is the one developed in the previous question. The alternative screen should show the latest system log messages, one per line (up to five lines in total). Whenever the user switches between screens by pressing the button, "next" should be displayed. Note: there should be support for a list of more than 5 messages, observing the simulated evolution of the logs over time.

Question 7: Following question 6, create an additional screen where the temperature graph over time can be viewed in real-time. The axes of the graph should be marked. Points should be presented starting from the origin, that is, $t=0$ represents the instant when the user "entered" the screen, with the temperature values obtained from then on represented in the graph. The functionality of rotating between screens using the push button should remain operational.

Question 8: Evolve the previous screen to also represent humidity in the same graph, but with a different color and a second ordinate axis. The functional characteristics are the same as those defined for question 7.

Question 9: Change the interface so that only the last x points that “fit” in the graph are represented in the temperature and humidity graphs. Note that if the user is viewing the default or log screen, the temperature and humidity data should continue to be produced at the same frequency and saved.

When returning to the graph display screen, the most recent x points recorded should be displayed, and the graph should be fully rendered with each new temperature and humidity measurement.